Master 2 TNAH, École nationale des chartes
20 octobre 2026

Marion Charpier



Introduction au langage de programmation Python

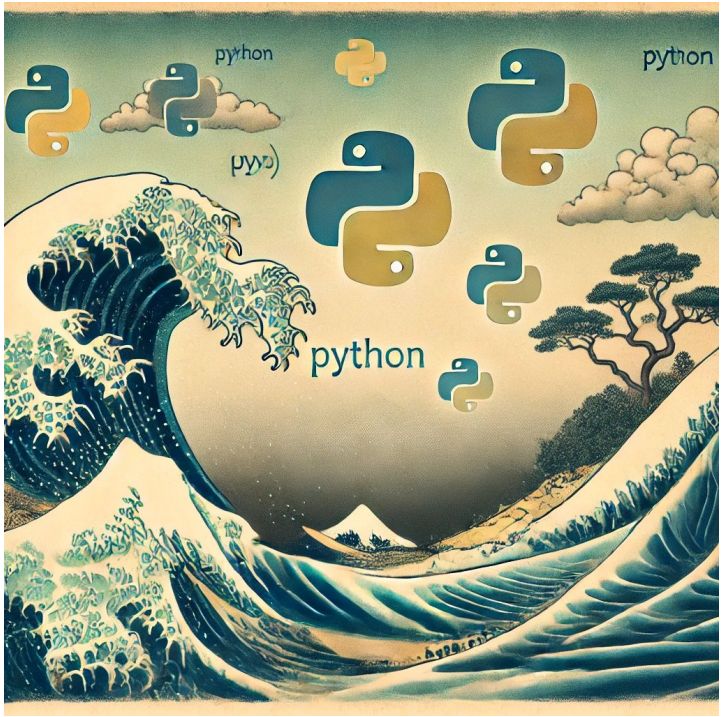
Séance 4 - Programmation orientée objet 2 : héritage et décorateurs

Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	14 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	3 novembre 2026	Du tableur à Python : CSV
7	10 novembre 2026	Explorer les collections numérisées : IIIF

Plan de la séance 4

1) **Correction de l'interrogation**
2) **L'héritage**
   a) Rappel : la classe Manuscrit
   b) Qu'est-ce que l'héritage ?
   c) Initialiser une classe enfant : super().__init__()
   d) Redéfinir une méthode : super()
3) **Typage et exceptions**
   a) Les annotations de type
   b) Vérifier un type et lever une exception : isinstance() et raise
   c) Gérer les exceptions : try et except
4) **Les décorateurs**
   a) Qu'est-ce qu'un décorateur ?
   b) Pourquoi utiliser des décorateurs ?
   c) Comment fonctionnent les décorateurs ?
   d) Rappel : *args et **kwargs



Rappel : la classe Manuscrit

```
import bestiaire

class Manuscrit:
    genre = "bestiaire"

    def __init__(self, cote, bibliotheque, date):
        self.cote = cote
        self.bibliotheque = bibliotheque
        self.date = date
        self.creatures = []

    def cote_complete(self):
        return f"{self.bibliotheque}, {self.cote}"

    def siecle(self):
        return bestiaire.siecle(self.date)

    def ajouter_creature(self, nom):
        self.creatures.append(nom)

    def fiche(self):
        return (f"{self.cote_complete()} ({self.genre})\n"
                f"  Siècle : {self.siecle()}\n"
                f"  Créatures : {self.creatures}")
```

Ce que contient la classe

- **Attribut de classe** : genre, partagé par tous les manuscrits.
- **__init__** : appelée automatiquement à l'instanciation, elle crée les **attributs d'instance**, propres à chaque objet.
- **self** : l'objet lui-même, que Python passe automatiquement.
- **Méthodes** : les actions de l'objet. Dans fiche(), self.cote_complete() appelle une autre méthode du même objet.

Instancier et utiliser

```
manuscrit = Manuscrit("MS 24", "Aberdeen, University Library", 1200)
manuscrit.ajouter_creature("licorne")
print(manuscrit.fiche())

# Aberdeen, University Library, MS 24 (bestiaire)
#   Siècle : XIIe siècle
#   Créatures : ['licorne']
```

L'héritage en programmation orientée objet



Définition

L'**héritage** permet à une classe, appelée **classe enfant** ou **sous-classe**, d'**hériter** des attributs et des méthodes d'une autre classe, appelée **classe parente** ou **super-classe**.

Syntaxe

On indique la classe parente entre parenthèses :

```
class ManuscritNumerise(Manuscrit):
    pass
```

La classe enfant sait déjà tout faire

Sans une ligne de code de plus, ManuscritNumerise possède les attributs et les méthodes de Manuscrit :

```
manuscrit_oxford = ManuscritNumerise("MS. Ashmole 1511",
"Oxford, Bodleian Library", 1210)
manuscrit_oxford.cote_complete()   # 'Oxford, Bodleian Library, MS.
Ashmole 1511'

manuscrit_oxford.siecle()          # 'XIIIe siècle'
manuscrit_oxford.genre             # 'bestiaire'
```

Intérêts

- **Réutiliser le code**
  - Les classes enfants utilisent les attributs et les méthodes de la classe parente, sans les recopier.
  - Une modification de la classe parente vaut pour toutes ses classes enfants.
- **Organiser les classes en hiérarchie**
  - L'héritage modélise une relation « est un » (*is a*) : un manuscrit numérisé **est un** manuscrit.
- **Étendre le code**
  - Une classe enfant peut **ajouter** des attributs et des méthodes, ou **modifier** celles qu'elle hérite, sans toucher à la classe parente.

Initialiser une classe enfant avec super().__init__()

À quoi sert super() ?

- super() permet d'appeler les méthodes de la **classe parente** depuis la classe enfant.
- super().__init__() appelle le __init__ de la classe parente : il initialise les **attributs hérités**.

Syntaxe

```
super().__init__(param1, param2, ...)
```

- On **n'écrit pas self** dans les arguments : Python le passe automatiquement, comme pour toute méthode.
- **Bonne pratique** : on l'appelle **en premier**, avant d'initialiser les attributs propres à la classe enfant.

Exemple : ajouter le manifeste IIIF

```
class ManuscritNumerise(Manuscrit):
    def __init__(self, cote, bibliotheque, date, manifeste):
        super().__init__(cote, bibliotheque, date)
        self.manifeste = manifeste
```

Explication

- **ManuscritNumerise(Manuscrit)** : la classe hérite de Manuscrit.
- **Paramètres** : cote, bibliotheque et date sont transmis à la classe parente ; manifeste est propre à la classe enfant.
- **super().__init__(cote, bibliotheque, date)** : le __init__ de Manuscrit crée self.cote, self.bibliotheque, self.date et la liste vide self.creatures.
- **self.manifeste = manifeste** : la classe enfant ajoute son propre attribut.

Redéfinir une méthode avec super()

Redéfinir une méthode
Une classe enfant peut écrire sa propre version d'une méthode héritée. Pour ses objets, Python utilise alors cette nouvelle version.

Utiliser super().methode_parente()

- Cela permet d'appeler, depuis la classe enfant, la version de la méthode écrite dans la classe parente.

Pourquoi utiliser super() ?

1. **Réutiliser le comportement de la classe parente** : on profite du code déjà écrit, sans le recopier.
2. **L'étendre ou le modifier** : on ajoute ce qui est propre à la classe enfant.

Exemple : la fiche d'un manuscrit numérisé

```
class ManuscritNumerise(Manuscrit):
    # __init__ : voir la slide 8

    def fiche(self):
        return f"{super().fiche()}\n  Manifeste : {self.manifeste}"
```

super().fiche() renvoie la fiche de Manuscrit, sur trois lignes. La classe enfant y ajoute une quatrième ligne :

```
Oxford, Bodleian Library, MS. Ashmole 1511 (bestiaire)
 Siècle : XIIIe siècle
 Créatures : []
 Manifeste : https://…
```

Attention : super().fiche() **renvoie** une valeur. Si on ne l'utilise pas, son résultat est perdu.

Les annotations de type



Un typage dynamique

Python est un langage **dynamiquement typé** : une variable n'a pas de type déclaré, et Python ne vérifie pas le type des arguments passés à une fonction. On peut toutefois **indiquer** le type attendu.

Les annotations de type

- Elles **indiquent** le type des paramètres et de la valeur renvoyée par une fonction.
- Elles sont **non contraignantes** : Python ne vérifie pas le type à l'exécution. Elles servent à la documentation et aux outils de vérification, comme ceux de VS Code.

Exemple : la fonction **siecle()**

```
def siecle(date: int) -> str:
    """Renvoyer le siècle d'une date."""
    if date <= 1200:
        resultat = "XIIe siècle"
    else:
        resultat = "XIIIe siècle"
    return resultat
```

- `date: int` indique que le paramètre **devrait** être un entier.
- `-> str` indique que la fonction **devrait** renvoyer une chaîne de caractères.

Python ne vérifie pas

```
siecle("vers 1210")
# TypeError: '<=' not supported between instances of 'str' and 'int'
```

Vérifier un type et lever une exception : isinstance() et raise



Qu'est-ce qu'une exception ?

Une **exception** est une erreur qui survient pendant l'exécution d'un programme et l'interrompt : TypeError, NameError, AttributeError…

Lever une exception avec raise

- raise déclenche **volontairement** une exception, avec un message clair.
- Syntaxe : raise NomDeLErreur("Message d'erreur")

Vérifier un type avec isinstance()

- isinstance(valeur, type) renvoie True si la valeur est du type indiqué, False sinon.
- Associée à raise, elle permet de **refuser** un argument du mauvais type.

Bibliothèques de validation

Pour des projets plus complexes, des bibliothèques comme **Pydantic** vérifient les types automatiquement.

Exemple : vérifier la date

```
def siecle(date: int) -> str:
    """Renvoyer le siècle d'une date."""
    if not isinstance(date, int):
        raise TypeError("la date doit être un nombre entier")
    if date <= 1200:
        resultat = "XIIe siècle"
    else:
        resultat = "XIIIe siècle"
    return resultat

siecle("vers 1210")
# TypeError: la date doit être un nombre entier
```

L'erreur est levée dès l'entrée dans la fonction, avec un message qui dit ce qui ne va pas.

À retenir

- Les annotations **indiquent** les types, mais ne les imposent pas.
- Pour les imposer : isinstance() et raise, ou une bibliothèque comme Pydantic.

Gérer les exceptions : try, except et exceptions personnalisées



Pourquoi gérer les exceptions ?

- Une exception non gérée **arrête** le programme.
- La gérer permet de **décider quoi faire** en cas d'erreur, et de continuer.

Gérer les exceptions avec try et except

- **Bloc try** : contient le code qui pourrait provoquer une exception.
- **Bloc except** : contient le code à exécuter si cette exception survient.

```
try:
    # Code susceptible de provoquer une exception
except NomDeLErreur:
    # Code à exécuter en cas d'exception
```

Créer ses propres exceptions

On peut utiliser les **exceptions natives** de Python, ou créer les siennes.

- **Définir une classe qui hérite de Exception** :

```
class DateInvalide(Exception):
    pass
```

Une DateInvalide **est une** Exception : elle hérite de tout son fonctionnement.

- **La lever** dans siecle() :

```
if not isinstance(date, int):
    raise DateInvalide("la date doit être un nombre entier")
```

- **La gérer** comme une exception native :

```
except DateInvalide:
```

Les décorateurs

Qu’est-ce qu’un décorateur ?

- Un **décorateur** est une fonction qui prend en entrée une autre fonction, et renvoie une nouvelle fonction au comportement modifié ou étendu.
- Il permet de modifier le comportement d’une fonction **sans changer son code source**.

Syntaxe avec @

```
def annoncer(fonction):
    def nouvelle_fonction():
        print("Avant l'exécution de la fonction")
        resultat = fonction()
        print("Après l'exécution de la fonction")
        return resultat
    return nouvelle_fonction

@annoncer
def afficher_titre():
    print(titre)

afficher_titre()
# Avant l'exécution de la fonction
# "III, 14. De la colombe"
# Après l'exécution de la fonction
```

Pourquoi utiliser des décorateurs ?



Ajouter une fonctionnalité sans modifier le code source

- Un décorateur enrichit une fonction existante sans toucher à son code.
- La fonction reste telle qu'elle a été écrite et testée : on ajoute, on ne modifie pas.

Séparer les tâches

- La fonction fait son travail : siecle() calcule un siècle.
- Le décorateur se charge d'une tâche annexe : annoncer l'appel, vérifier les arguments, gérer les erreurs.
- Le code est plus lisible : chaque partie a un seul rôle.

Réutiliser le code

- Un même décorateur s'applique à plusieurs fonctions: @annoncer peut décorer afficher_titre() comme siecle(), sans dupliquer le code.
- Pour modifier le comportement ajouté, on ne change que le décorateur : toutes les fonctions décorées en profitent.

Comment fonctionnent les décorateurs ?

Fonctionnement

- Le décorateur **enveloppe** la fonction d'origine : il exécute du code **avant** et **après** elle.

Application du décorateur

- On écrit @nom_du_decorateur juste au-dessus de la définition de la fonction.

Exécution

- Quand on appelle siecle(), c'est en réalité nouvelle_fonction() qui s'exécute.
- Elle exécute le code « avant », puis la fonction d'origine, puis le code « après », et renvoie le résultat.

Exemple : décorer une fonction qui a des paramètres

```
def annoncer(fonction):
    def nouvelle_fonction(*args, **kwargs):
        print("Avant l'exécution de la fonction")
        resultat = fonction(*args, **kwargs)
        print("Après l'exécution de la fonction")
        return resultat
    return nouvelle_fonction

@annoncer
def siecle(date: int) -> str:
    ...

print(siecle(1210))
# Avant l'exécution de la fonction
# Après l'exécution de la fonction
# XIIIe siècle
```

Grâce à *args et **kwargs, nouvelle_fonction() accepte les arguments de n'importe quelle fonction et les transmet à la fonction d'origine.

Rappel : *args et **kwargs dans les décorateurs

Rappel de la séance 2

***args** : regroupe les arguments **positionnels** supplémentaires dans un **tuple**.

```
def decrire_creatures(cote, *noms):
    return f"{cote} : {len(noms)} créatures {noms}"

decrire_creatures("MS 24", "dragon", "licorne", "phénix")
# 'MS 24 : 3 créatures ('dragon', 'licorne', 'phénix')'
```

****kwargs** : regroupe les arguments **nommés** supplémentaires dans un **dictionnaire**.

```
def notice_libre(**infos):
    return infos

notice_libre(cote="MS 24", date=1200, numerise=True)
# {'cote': 'MS 24', 'date': 1200, 'numerise': True}
```

Utilisation dans les décorateurs

```
def nouvelle_fonction(*args, **kwargs):
    ...
    resultat = fonction(*args, **kwargs)
```

- Dans le def, les étoiles **regroupent** tous les arguments reçus, quels qu'ils soient.
- À l'appel de fonction, elles les **déballent** pour les transmettre tels quels.
- Le décorateur fonctionne donc avec **toutes les fonctions**, quelles que soient leurs signatures : afficher_titre() sans argument, siecle(1210) avec un argument.

Notions clés à retenir

Héritage

- Une classe enfant **hérite** des attributs et des méthodes de sa classe parente : class ManuscritNumerise(Manuscrit).
- Il modélise une relation « **est un** » : un manuscrit numérisé est un manuscrit.
- **super()** appelle les méthodes de la classe parente :
  - super().__init__() initialise les attributs hérités ;
  - super().fiche() réutilise une méthode qu'on redéfinit, à condition d'utiliser ce qu'elle **renvoie**.

Typage et exceptions

- Les **annotations de type** (date: int, -> str) **indiquent** les types, mais Python ne les vérifie pas.
- **isinstance() et raise** permettent de refuser un argument du mauvais type, avec un message clair.
- **try et except** gèrent une exception sans arrêter le programme.
- Une **exception personnalisée** est une classe qui hérite de Exception.

Décorateurs

- Un décorateur est une fonction qui **modifie ou étend** le comportement d'une autre fonction, sans en changer le code source.
- Il s'applique avec @ au-dessus de la définition de la fonction.
- Pourquoi les utiliser ? Pour **ajouter une fonctionnalité** sans toucher à la fonction, **séparer les tâches** et **réutiliser** le même code sur plusieurs fonctions.

***args et **kwargs**

- *args accepte un nombre variable d'arguments **positionnels** (tuple), **kwargs d'arguments **nommés** (dictionnaire).
- Dans un décorateur, ils permettent d'envelopper **n'importe quelle fonction**, quelle que soit sa signature.